

(Explorty Data Enalysis)

***** EDA *****

Prject objective

1 Sales Performance Analysis

-- Analysis overall sales perforamnce and revenue treands

2 Customer Behavior Analysis

-- understand Customer purchasing behaviors and spending patterns

3 Customer Satisfactions Analysis

-- evaluate customers satisficationn usinf reviews and delivery performance

4 Delivery Performance Analysis

-- mesure delivery efficiency and identify logistics issue

5 Product Category Analysis

-- identify high-performing and low-proforming product category

6 Sallers Perforamnce Analysis

-- evaluate sellers contributions and services quality

7 Payment Analysis

-- Analysis customers payments behaviors and payment trends

8 Cancellations Analysis

-- identify couese of orders cancelations

9 Root Cause Analysis

-- identify the underlying cause affecting customers experience and business growth

Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import FuncFormatter
from sqlalchemy import create_engine
import warnings
warnings.filterwarnings('ignore')
```

importing datasets

```
customers_df = pd.read_csv(r"D:\build project\Olist_project\Data\olist_customers_dataset.csv")
geolocations_df = pd.read_csv(r"D:\build project\Olist_project\Data\olist_geolocation_dataset.csv")
order_items_df = pd.read_csv(r"D:\build project\Olist_project\Data\olist_order_items_dataset.csv")
order_payments_df = pd.read_csv(r"D:\build project\Olist_project\Data\olist_order_payments_dataset.csv")
order_reviews_df = pd.read_csv(r"D:\build project\Olist_project\Data\olist_order_reviews_dataset.csv")
orders_df = pd.read_csv(r"D:\build project\Olist_project\Data\olist_orders_dataset.csv")
products_df = pd.read_csv(r"D:\build project\Olist_project\Data\olist_products_dataset.csv")
sellers_df = pd.read_csv(r"D:\build project\Olist_project\Data\olist_sellers_dataset.csv")
transactions_df = pd.read_csv(r"D:\build project\Olist_project\Data\product_category_name_translation.csv")

tables = {
    "customers_df":customers_df,
    "geolocations_df":geolocations_df,
    "order_items_df":order_items_df,
    "order_payments_df":order_payments_df,
    "order_reviews_df":order_reviews_df,
    "orders_df":orders_df,
    "products_df":products_df,
    "sellers_df":sellers_df,
    "transactions_df":transactions_df
}

for name,tab in tables.items():
    print(f'{name}')
    print(f'Rows:{tab.shape[0]} Columns: {tab.shape[1]}')
    print(25*'-')
```

```

customers_df
Rows:99441 Columns: 5
-----

geolocations_df
Rows:1000163 Columns: 5
-----

order_items_df
Rows:112650 Columns: 7
-----

order_payments_df
Rows:103886 Columns: 5
-----

order_reviews_df
Rows:99224 Columns: 7
-----

orders_df
Rows:99441 Columns: 8
-----

prducts_df
Rows:32951 Columns: 9
-----

sallers_df
Rows:3095 Columns: 4
-----

transactions_df
Rows:71 Columns: 2
-----

for i,j in tables.items():
    print("-----",i,"-----")
    print(j.columns)
    print(50*'-')

----- customers_df -----
Index(['customer_id', 'customer_unique_id', 'customer_zip_code_prefix',
      'customer_city', 'customer_state'],
      dtype='object')
-----

----- geolocations_df -----
Index(['geolocation_zip_code_prefix', 'geolocation_lat', 'geolocation_lng',
      'geolocation_city', 'geolocation_state'],
      dtype='object')
-----

----- order_items_df -----
Index(['order_id', 'order_item_id', 'product_id', 'seller_id',
      'shipping_limit_date', 'price', 'freight_value'],
      dtype='object')

```

```

-----
----- order_payments_df -----
Index(['order_id', 'payment_sequential', 'payment_type',
      'payment_installments', 'payment_value'],
      dtype='object')
-----
----- order_reviews_df -----
Index(['review_id', 'order_id', 'review_score', 'review_comment_title',
      'review_comment_message', 'review_creation_date',
      'review_answer_timestamp'],
      dtype='object')
-----
----- orders_df -----
Index(['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp',
      'order_approved_at', 'order_delivered_carrier_date',
      'order_delivered_customer_date', 'order_estimated_delivery_date'],
      dtype='object')
-----
----- products_df -----
Index(['product_id', 'product_category_name', 'product_name_lenght',
      'product_description_lenght', 'product_photos_qty', 'product_weight_g',
      'product_length_cm', 'product_height_cm', 'product_width_cm'],
      dtype='object')
-----
----- sellers_df -----
Index(['seller_id', 'seller_zip_code_prefix', 'seller_city', 'seller_state'], dtype='object')
-----
----- transactions_df -----
Index(['product_category_name', 'product_category_name_english'], dtype='object')
-----

```

create relationship (merge) dataset

```

df_train = orders_df.merge(order_items_df, on='order_id', how='outer')
df_train = df_train.merge(products_df, on='product_id', how='outer')
df_train = df_train.merge(order_payments_df, on='order_id', how='outer', validate='m:m')
df_train = df_train.merge(customers_df, on='customer_id', how='outer')
df_train = df_train.merge(order_reviews_df, on='order_id', how='outer')
df_train = df_train.merge(sellers_df, on='seller_id', how='outer')

df_train.head()

```

	order_id	customer_id \
0	7f39ba4c9052be115350065d07583cac	d7fc82cbeafea77bd0a8fbbf6296e387
1	9dc8d1a6f16f1b89874c29c9d8d30447	d9442164acf4b03109425633efaa0cfc
2	d455a8cb295653b55abda06d434ab492	944b72539d7e1f7f7fc6e46639ef1fe3

```

3 006e43460a55bc60c0a437521e426529 23bfd4316e261786deed5a08231c75bc
4 00dfb074b5c910fbd08e04691c4b712f a5ced4926d7d8fa71e9be2b007720356

```

```

order_status order_purchase_timestamp order_approved_at \
0 delivered 2017-10-18 08:16:34 2017-10-18 23:56:20
1 delivered 2017-10-12 13:33:22 2017-10-12 13:49:22
2 delivered 2017-09-26 22:17:05 2017-09-27 22:24:16
3 delivered 2017-05-11 00:24:35 2017-05-11 01:30:22
4 delivered 2017-06-08 19:43:35 2017-06-08 19:55:19

```

```

order_delivered_carrier_date order_delivered_customer_date \
0 2017-10-20 14:29:01 2017-10-27 16:46:05
1 2017-10-17 15:42:42 2017-10-24 20:17:44
2 2017-09-29 15:53:03 2017-10-07 16:12:47
3 2017-05-12 12:26:32 2017-05-19 09:51:17
4 2017-06-09 15:12:41 2017-06-15 09:03:59

```

```

order_estimated_delivery_date order_item_id \
0 2017-11-09 00:00:00 1.0
1 2017-11-06 00:00:00 1.0
2 2017-10-30 00:00:00 1.0
3 2017-06-02 00:00:00 1.0
4 2017-07-10 00:00:00 1.0

```

```

product_id ... customer_state \
0 a2ff5a97bf95719e38ea2e3b4105bce8 ... MG
1 a2ff5a97bf95719e38ea2e3b4105bce8 ... MG
2 a2ff5a97bf95719e38ea2e3b4105bce8 ... PR
3 08574b074924071f4e201e151b152b4e ... SP
4 4f3b83b83f7fb280fca4f90ba502f50e ... SP

```

```

review_id review_score review_comment_title \
0 2abb25fde5aafe9bc85d031dffb2f4ac 1.0 NaN
1 799c3e3362ac0d09cfa1b39b51647490 5.0 NaN
2 f6923b4d17e0f4e87b38cb26c758d3c6 5.0 NaN
3 7f8c3db58f0112378d38f6065d6c3e33 5.0 NaN
4 c7ae6b7ce2981450a84ea641f5b64dc0 5.0 NaN

```

```

review_comment_message review_creation_date \
0 Produto preto, reembalado na caixa de um branco... 2017-10-28 00:00:00
1 NaN 2017-10-25 00:00:00
2 NaN 2017-10-08 00:00:00
3 NaN 2017-05-20 00:00:00
4 De qualidade e entrega antes do prazo.\r\nReco... 2017-06-16 00:00:00

```

```

review_answer_timestamp seller_zip_code_prefix seller_city seller_state

```

0	2017-10-28 14:32:42	9080.0	santo andre	SP
1	2017-10-25 23:15:15	9080.0	santo andre	SP
2	2017-10-11 21:42:45	9080.0	santo andre	SP
3	2017-05-21 12:35:51	29156.0	cariacica	ES
4	2017-06-16 22:47:27	29156.0	cariacica	ES

[5 rows x 39 columns]

Data validations‘

```
df_train.shape
```

```
(119143, 39)
```

```
df_train.columns
```

```
Index(['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp',
      'order_approved_at', 'order_delivered_carrier_date',
      'order_delivered_customer_date', 'order_estimated_delivery_date',
      'order_item_id', 'product_id', 'seller_id', 'shipping_limit_date',
      'price', 'freight_value', 'product_category_name',
      'product_name_lenght', 'product_description_lenght',
      'product_photos_qty', 'product_weight_g', 'product_length_cm',
      'product_height_cm', 'product_width_cm', 'payment_sequential',
      'payment_type', 'payment_installments', 'payment_value',
      'customer_unique_id', 'customer_zip_code_prefix', 'customer_city',
      'customer_state', 'review_id', 'review_score', 'review_comment_title',
      'review_comment_message', 'review_creation_date',
      'review_answer_timestamp', 'seller_zip_code_prefix', 'seller_city',
      'seller_state'],
      dtype='object')
```

```
df_train.dtypes
```

order_id	object
customer_id	object
order_status	object
order_purchase_timestamp	object
order_approved_at	object
order_delivered_carrier_date	object
order_delivered_customer_date	object
order_estimated_delivery_date	object
order_item_id	float64
product_id	object
seller_id	object
shipping_limit_date	object

```

price float64
freight_value float64
product_category_name object
product_name_lenght float64
product_description_lenght float64
product_photos_qty float64
product_weight_g float64
product_length_cm float64
product_height_cm float64
product_width_cm float64
payment_sequential float64
payment_type object
payment_installments float64
payment_value float64
customer_unique_id object
customer_zip_code_prefix int64
customer_city object
customer_state object
review_id object
review_score float64
review_comment_title object
review_comment_message object
review_creation_date object
review_answer_timestamp object
seller_zip_code_prefix float64
seller_city object
seller_state object
dtype: object

```

```
df_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 119143 entries, 0 to 119142
```

```
Data columns (total 39 columns):
```

#	Column	Non-Null Count	Dtype
0	order_id	119143 non-null	object
1	customer_id	119143 non-null	object
2	order_status	119143 non-null	object
3	order_purchase_timestamp	119143 non-null	object
4	order_approved_at	118966 non-null	object
5	order_delivered_carrier_date	117057 non-null	object
6	order_delivered_customer_date	115722 non-null	object
7	order_estimated_delivery_date	119143 non-null	object
8	order_item_id	118310 non-null	float64
9	product_id	118310 non-null	object
10	seller_id	118310 non-null	object

11	shipping_limit_date	118310	non-null	object
12	price	118310	non-null	float64
13	freight_value	118310	non-null	float64
14	product_category_name	116601	non-null	object
15	product_name_lenght	116601	non-null	float64
16	product_description_lenght	116601	non-null	float64
17	product_photos_qty	116601	non-null	float64
18	product_weight_g	118290	non-null	float64
19	product_length_cm	118290	non-null	float64
20	product_height_cm	118290	non-null	float64
21	product_width_cm	118290	non-null	float64
22	payment_sequential	119140	non-null	float64
23	payment_type	119140	non-null	object
24	payment_installments	119140	non-null	float64
25	payment_value	119140	non-null	float64
26	customer_unique_id	119143	non-null	object
27	customer_zip_code_prefix	119143	non-null	int64
28	customer_city	119143	non-null	object
29	customer_state	119143	non-null	object
30	review_id	118146	non-null	object
31	review_score	118146	non-null	float64
32	review_comment_title	13989	non-null	object
33	review_comment_message	50245	non-null	object
34	review_creation_date	118146	non-null	object
35	review_answer_timestamp	118146	non-null	object
36	seller_zip_code_prefix	118310	non-null	float64
37	seller_city	118310	non-null	object
38	seller_state	118310	non-null	object

dtypes: float64(15), int64(1), object(23)

memory usage: 35.5+ MB

df_train.describe().T

	count	mean	std	min \
order_item_id	118310.0	1.196543	0.699489	1.00
price	118310.0	120.646603	184.109691	0.85
freight_value	118310.0	20.032387	15.836850	0.00
product_name_lenght	116601.0	48.767498	10.033540	5.00
product_description_lenght	116601.0	785.967822	652.584121	4.00
product_photos_qty	116601.0	2.205161	1.717452	1.00
product_weight_g	118290.0	2112.250740	3786.695111	0.00
product_length_cm	118290.0	30.265145	16.189367	7.00
product_height_cm	118290.0	16.619706	13.453584	2.00
product_width_cm	118290.0	23.074799	11.749139	6.00
payment_sequential	119140.0	1.094737	0.730141	1.00

payment_installments	119140.0	2.941246	2.777848	0.00
payment_value	119140.0	172.735135	267.776077	0.00
customer_zip_code_prefix	119143.0	35033.451298	29823.198969	1003.00
review_score	118146.0	4.015582	1.400436	1.00
seller_zip_code_prefix	118310.0	24442.410413	27573.004511	1001.00

	25%	50%	75%	max
order_item_id	1.00	1.00	1.00	21.00
price	39.90	74.90	134.90	6735.00
freight_value	13.08	16.28	21.18	409.68
product_name_lenght	42.00	52.00	57.00	76.00
product_description_lenght	346.00	600.00	983.00	3992.00
product_photos_qty	1.00	1.00	3.00	20.00
product_weight_g	300.00	700.00	1800.00	40425.00
product_length_cm	18.00	25.00	38.00	105.00
product_height_cm	8.00	13.00	20.00	105.00
product_width_cm	15.00	20.00	30.00	118.00
payment_sequential	1.00	1.00	1.00	29.00
payment_installments	1.00	2.00	4.00	24.00
payment_value	60.85	108.16	189.24	13664.08
customer_zip_code_prefix	11250.00	24240.00	58475.00	99990.00
review_score	4.00	5.00	5.00	5.00
seller_zip_code_prefix	6429.00	13660.00	27972.00	99730.00

df_train.isnull().sum()

order_id	0
customer_id	0
order_status	0
order_purchase_timestamp	0
order_approved_at	177
order_delivered_carrier_date	2086
order_delivered_customer_date	3421
order_estimated_delivery_date	0
order_item_id	833
product_id	833
seller_id	833
shipping_limit_date	833
price	833
freight_value	833
product_category_name	2542
product_name_lenght	2542
product_description_lenght	2542
product_photos_qty	2542
product_weight_g	853
product_length_cm	853
product_height_cm	853

```

product_width_cm      853
payment_sequential     3
payment_type           3
payment_installments   3
payment_value          3
customer_unique_id     0
customer_zip_code_prefix 0
customer_city          0
customer_state         0
review_id              997
review_score            997
review_comment_title    105154
review_comment_message  68898
review_creation_date     997
review_answer_timestamp  997
seller_zip_code_prefix   833
seller_city             833
seller_state            833
dtype: int64

df_train.duplicated().sum()

np.int64(0)

df_train.isnull().sum()/len(df_train)*100

order_id      0.000000
customer_id   0.000000
order_status  0.000000
order_purchase_timestamp  0.000000
order_approved_at  0.148561
order_delivered_carrier_date  1.750837
order_delivered_customer_date  2.871339
order_estimated_delivery_date  0.000000
order_item_id  0.699160
product_id    0.699160
seller_id     0.699160
shipping_limit_date  0.699160
price         0.699160
freight_value  0.699160
product_category_name  2.133571
product_name_lenght    2.133571
product_description_lenght  2.133571
product_photos_qty     2.133571
product_weight_g       0.715946
product_length_cm      0.715946
product_height_cm      0.715946
product_width_cm       0.715946

```

```

payment_sequential          0.002518
payment_type                0.002518
payment_installments        0.002518
payment_value               0.002518
customer_unique_id          0.000000
customer_zip_code_prefix    0.000000
customer_city               0.000000
customer_state              0.000000
review_id                   0.836810
review_score                0.836810
review_comment_title        88.258647
review_comment_message      57.827988
review_creation_date        0.836810
review_answer_timestamp     0.836810
seller_zip_code_prefix      0.699160
seller_city                 0.699160
seller_state                0.699160
dtype: float64

```

handake missing values

```

df_train.drop(columns=['review_comment_title', 'review_comment_message'], axis=1, inplace= True)
df_train.drop(columns=['product_name_lenght', 'product_description_lenght', 'product_photos_qty'], axis=1, inplace= True)
mask = (
    (df_train['order_status'] == 'delivered') &
    (df_train['order_delivered_customer_date'].isnull())
)

```

```

df_train.loc[
    mask,
    'order_delivered_customer_date'
] = df_train.loc[
    mask,
    'order_estimated_delivery_date'
]

```

```

df_train = df_train.dropna(subset=['order_item_id'])
df_train = df_train.dropna(subset=['payment_type'])
df_train = df_train.dropna(subset=['review_id'])
df_train['product_weight_g'] = df_train['product_weight_g'].fillna(df_train['product_weight_g'].mean())
df_train['product_length_cm'] = df_train['product_length_cm'].fillna(df_train['product_length_cm'].mean())
df_train['product_height_cm'] = df_train['product_height_cm'].fillna(df_train['product_height_cm'].mean())
df_train['product_width_cm'] = df_train['product_width_cm'].fillna(df_train['product_width_cm'].mean())
df_train['product_category_name'] = df_train['product_category_name'].fillna('Unknow')
df_train.isnull().sum()

```

```

order_id          0
customer_id       0

```

order_status	0
order_purchase_timestamp	0
order_approved_at	15
order_delivered_carrier_date	1235
order_delivered_customer_date	2463
order_estimated_delivery_date	0
order_item_id	0
product_id	0
seller_id	0
shipping_limit_date	0
price	0
freight_value	0
product_category_name	0
product_weight_g	0
product_length_cm	0
product_height_cm	0
product_width_cm	0
payment_sequential	0
payment_type	0
payment_installments	0
payment_value	0
customer_unique_id	0
customer_zip_code_prefix	0
customer_city	0
customer_state	0
review_id	0
review_score	0
review_creation_date	0
review_answer_timestamp	0
seller_zip_code_prefix	0
seller_city	0
seller_state	0
dtype:	int64

Convert date and time

```

df_train['order_purchase_timestamp'] = pd.to_datetime(df_train['order_purchase_timestamp'])
df_train['order_approved_at'] = pd.to_datetime(df_train['order_approved_at'])
df_train['order_delivered_carrier_date'] = pd.to_datetime(df_train['order_delivered_carrier_date'])
df_train['order_delivered_customer_date'] = pd.to_datetime(df_train['order_delivered_customer_date'])
df_train['order_estimated_delivery_date'] = pd.to_datetime(df_train['order_estimated_delivery_date'])
df_train['shipping_limit_date'] = pd.to_datetime(df_train['shipping_limit_date'])
df_train['review_creation_date'] = pd.to_datetime(df_train['review_creation_date'])
df_train['review_answer_timestamp'] = pd.to_datetime(df_train['review_answer_timestamp'])
df_train['order_item_id'] = df_train['order_item_id'].astype(int)
df_train['payment_sequential'] = df_train['payment_sequential'].astype(int)
df_train['payment_installments'] = df_train['payment_installments'].astype(int)

```

```
df_train['seller_zip_code_prefix'] = df_train['seller_zip_code_prefix'].astype(int)
#df_train['delivery_days'] = df_train['delivery_days'].astype(int)
```

Feature Engneearing

```
# create column how mant time dilever the dilevery
df_train['delivery_days'] = (df_train['order_delivered_customer_date'] - df_train['order_purchase_timestamp']).dt.days

# create column how many late dilevery
df_train['delivery_delay_days'] = (df_train['order_delivered_customer_date'] - df_train['order_estimated_delivery_date']).dt.days

# create column only check late dilevery and not late dilevery (1-- yes & 0-- NO)
df_train['delay_flag'] = (
    df_train['order_delivered_customer_date'] > df_train['order_estimated_delivery_date'])

# create column purchase month
df_train['purchase_month'] = (df_train['order_purchase_timestamp'].dt.strftime('%b'))

# create column purchase year
df_train['purchase_year'] = (df_train['order_purchase_timestamp'].dt.year)

df_train.head(3)
```

	order_id	customer_id \
0	7f39ba4c9052be115350065d07583cac	d7fc82cbeafea77bd0a8fbbf6296e387
1	9dc8d1a6f16f1b89874c29c9d8d30447	d9442164acf4b03109425633efaa0cfc
2	d455a8cb295653b55abda06d434ab492	944b72539d7e1f7f7fc6e46639ef1fe3

	order_status	order_purchase_timestamp	order_approved_at \
0	delivered	2017-10-18 08:16:34	2017-10-18 23:56:20
1	delivered	2017-10-12 13:33:22	2017-10-12 13:49:22
2	delivered	2017-09-26 22:17:05	2017-09-27 22:24:16

	order_delivered_carrier_date	order_delivered_customer_date \
0	2017-10-20 14:29:01	2017-10-27 16:46:05
1	2017-10-17 15:42:42	2017-10-24 20:17:44
2	2017-09-29 15:53:03	2017-10-07 16:12:47

	order_estimated_delivery_date	order_item_id \
0	2017-11-09	1
1	2017-11-06	1
2	2017-10-30	1

	product_id	... review_creation_date \
0	a2ff5a97bf95719e38ea2e3b4105bce8	... 2017-10-28
1	a2ff5a97bf95719e38ea2e3b4105bce8	... 2017-10-25

```

2 a2ff5a97bf95719e38ea2e3b4105bce8 ... 2017-10-08

review_answer_timestamp seller_zip_code_prefix seller_city seller_state \
0 2017-10-28 14:32:42 9080 santo andre SP
1 2017-10-25 23:15:15 9080 santo andre SP
2 2017-10-11 21:42:45 9080 santo andre SP

delivery_days delivery_delay_days delay_flag purchase_month \
0 9.0 -13.0 0 Oct
1 12.0 -13.0 0 Oct
2 10.0 -23.0 0 Sep

purchase_year
0 2017
1 2017
2 2017

[3 rows x 39 columns]

```

EDA (Explorty Data Analysis)

```
df_train.shape
```

```
(117329, 39)
```

```
df_train.describe().T
```

```

count mean \
order_purchase_timestamp 117329 2017-12-30 10:29:39.749081344
order_approved_at 117314 2017-12-30 21:58:19.189721344
order_delivered_carrier_date 116094 2018-01-03 12:09:47.466819584
order_delivered_customer_date 114866 2018-01-12 23:51:30.445336064
order_estimated_delivery_date 117329 2018-01-23 07:08:36.977047296
order_item_id 117329.0 1.194121
shipping_limit_date 117329 2018-01-06 02:01:32.273274624
price 117329.0 120.524349
freight_value 117329.0 20.027514
product_weight_g 117329.0 2110.763062
product_length_cm 117329.0 30.254456
product_height_cm 117329.0 16.612476
product_width_cm 117329.0 23.071452
payment_sequential 117329.0 1.094452
payment_installments 117329.0 2.940151
payment_value 117329.0 172.062565
customer_zip_code_prefix 117329.0 35060.118112

```

review_score	117329.0	4.031467
review_creation_date	117329	2018-01-11 23:57:07.684545280
review_answer_timestamp	117329	2018-01-15 03:28:45.726342144
seller_zip_code_prefix	117329.0	24450.781955
delivery_days	114866.0	11.980177
delivery_delay_days	114866.0	-12.083732
delay_flag	117329.0	0.075514
purchase_year	117329.0	2017.537591

		min \
order_purchase_timestamp	2016-09-04 21:15:19	
order_approved_at	2016-10-04 09:43:32	
order_delivered_carrier_date	2016-10-08 10:34:01	
order_delivered_customer_date	2016-10-11 13:46:32	
order_estimated_delivery_date	2016-10-20 00:00:00	
order_item_id	1.0	
shipping_limit_date	2016-09-19 00:15:34	
price	0.85	
freight_value	0.0	
product_weight_g	0.0	
product_length_cm	7.0	
product_height_cm	2.0	
product_width_cm	6.0	
payment_sequential	1.0	
payment_installments	0.0	
payment_value	0.0	
customer_zip_code_prefix	1003.0	
review_score	1.0	
review_creation_date	2016-10-15 00:00:00	
review_answer_timestamp	2016-10-16 03:20:17	
seller_zip_code_prefix	1001.0	
delivery_days	0.0	
delivery_delay_days	-147.0	
delay_flag	0.0	
purchase_year	2016.0	

		25% \
order_purchase_timestamp	2017-09-11 16:37:08	
order_approved_at	2017-09-12 04:35:32.500000	
order_delivered_carrier_date	2017-09-14 20:27:55.249999872	
order_delivered_customer_date	2017-09-23 01:23:34	
order_estimated_delivery_date	2017-10-03 00:00:00	
order_item_id	1.0	
shipping_limit_date	2017-09-18 15:55:47	
price	39.9	
freight_value	13.08	

product_weight_g	300.0
product_length_cm	18.0
product_height_cm	8.0
product_width_cm	15.0
payment_sequential	1.0
payment_installments	1.0
payment_value	60.75
customer_zip_code_prefix	11250.0
review_score	4.0
review_creation_date	2017-09-22 00:00:00
review_answer_timestamp	2017-09-25 23:24:55
seller_zip_code_prefix	6429.0
delivery_days	6.0
delivery_delay_days	-17.0
delay_flag	0.0
purchase_year	2017.0

50% \

order_purchase_timestamp	2018-01-18 09:42:57
order_approved_at	2018-01-18 14:16:46
order_delivered_carrier_date	2018-01-23 18:50:08
order_delivered_customer_date	2018-02-01 16:25:23.500000
order_estimated_delivery_date	2018-02-14 00:00:00
order_item_id	1.0
shipping_limit_date	2018-01-25 09:09:32
price	74.9
freight_value	16.28
product_weight_g	700.0
product_length_cm	25.0
product_height_cm	13.0
product_width_cm	20.0
payment_sequential	1.0
payment_installments	2.0
payment_value	108.1
customer_zip_code_prefix	24240.0
review_score	5.0
review_creation_date	2018-02-01 00:00:00
review_answer_timestamp	2018-02-03 23:11:32
seller_zip_code_prefix	13660.0
delivery_days	10.0
delivery_delay_days	-13.0
delay_flag	0.0
purchase_year	2018.0

75% \

order_purchase_timestamp	2018-05-03 19:53:54
--------------------------	---------------------

order_approved_at	2018-05-04 04:11:43.500000
order_delivered_carrier_date	2018-05-07 15:25:00
order_delivered_customer_date	2018-05-15 14:12:41
order_estimated_delivery_date	2018-05-25 00:00:00
order_item_id	1.0
shipping_limit_date	2018-05-10 03:31:06
price	134.9
freight_value	21.18
product_weight_g	1800.0
product_length_cm	38.0
product_height_cm	20.0
product_width_cm	30.0
payment_sequential	1.0
payment_installments	4.0
payment_value	189.06
customer_zip_code_prefix	58770.0
review_score	5.0
review_creation_date	2018-05-15 00:00:00
review_answer_timestamp	2018-05-17 16:20:40
seller_zip_code_prefix	28035.0
delivery_days	15.0
delivery_delay_days	-7.0
delay_flag	0.0
purchase_year	2018.0

	max	std
order_purchase_timestamp	2018-09-03 09:06:57	NaN
order_approved_at	2018-09-03 17:40:06	NaN
order_delivered_carrier_date	2018-09-11 19:48:28	NaN
order_delivered_customer_date	2018-10-17 13:22:46	NaN
order_estimated_delivery_date	2018-10-25 00:00:00	NaN
order_item_id	21.0	0.684225
shipping_limit_date	2020-04-09 22:35:08	NaN
price	6735.0	182.944843
freight_value	409.68	15.828077
product_weight_g	40425.0	3784.806306
product_length_cm	105.0	16.17614
product_height_cm	105.0	13.451478
product_width_cm	118.0	11.744873
payment_sequential	29.0	0.731174
payment_installments	24.0	2.77537
payment_value	13664.08	265.388194
customer_zip_code_prefix	99990.0	29849.496175
review_score	5.0	1.387927
review_creation_date	2018-08-31 00:00:00	NaN
review_answer_timestamp	2018-10-29 12:27:35	NaN

seller_zip_code_prefix	99730.0	27582.364358
delivery_days	208.0	9.371545
delivery_delay_days	188.0	10.094604
delay_flag	1.0	0.26422
purchase_year	2018.0	0.505092

df_train.describe(include='object').T

	count	unique	top	freq
order_id	117329	97916	895ab968e7bb0d5659d16cd74cd1650c	63
customer_id	117329	97916	270c23a11d024a44c896d1894b261a83	63
order_status	117329	7	delivered	114859
product_id	117329	32789	aca2eb7d00ea1a7b8ebd4e68314663af	533
seller_id	117329	3090	4a3ca9315b744ce9f8e9374361493884	2128
product_category_name	117329	74	cama_mesa_banho	11847
payment_type	117329	4	credit_card	86520
customer_unique_id	117329	94720	9a736b248f67d166d2fbb006bcb877c3	75
customer_city	117329	4108	sao paulo	18582
customer_state	117329	27	SP	49527
review_id	117329	97708	eef5dbca8d37dfce6db7d7b16dd0525e	63
seller_city	117329	611	sao paulo	29048
seller_state	117329	23	SP	83671
purchase_month	117329	12	Aug	12597

df_train.corr(numeric_only=True)['review_score']

order_item_id	-0.129344
price	-0.004515
freight_value	-0.037076
product_weight_g	-0.028638
product_length_cm	-0.020439
product_height_cm	-0.026407
product_width_cm	-0.010742
payment_sequential	-0.004634
payment_installments	-0.042279
payment_value	-0.082404
customer_zip_code_prefix	-0.023349
review_score	1.000000
seller_zip_code_prefix	0.026143
delivery_days	-0.302508
delivery_delay_days	-0.228202
delay_flag	-0.305668
purchase_year	-0.006575

Name: review_score, dtype: float64

formate convertings

```
def format_k(value, pos):
    if value >= 1000000:
        return f'{value/1000000:.1f}M'
    elif value >= 1000:
        return f'{value/1000:.0f}K'
    else:
        return str(int(value))
```

Bussiness problem solve (Sales Analysis)

```
total_revenue = df_train['payment_value'].sum()
print(f'Total Revenue {total_revenue}')
Total Revenue 20187928.7
```

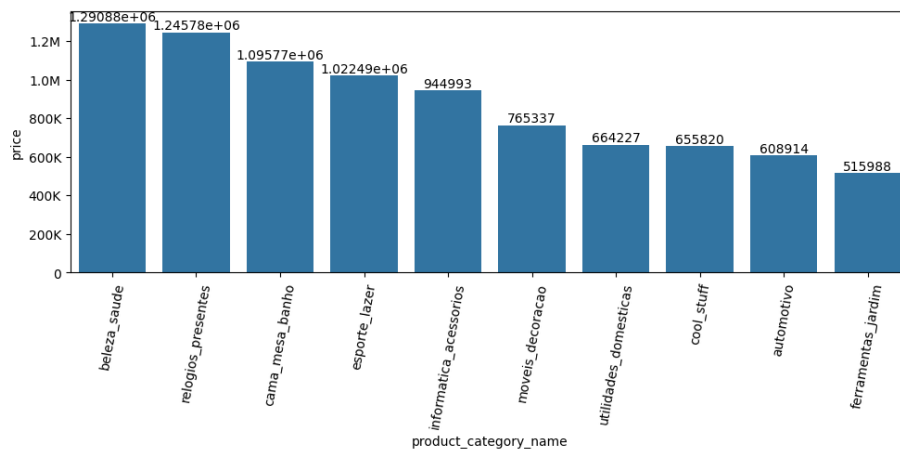
Top Product categories

```
top_categories = df_train.groupby(
    'product_category_name'
)['price'].sum().sort_values(ascending = False).head(10)
top_categories = top_categories.reset_index()
plt.figure(figsize=(10,5))

ax = sns.barplot(x='product_category_name', y='price', data=top_categories)
plt.xticks(rotation=80)
ax.yaxis.set_major_formatter(FuncFormatter(format_k))

for bar in ax.containers:
    ax.bar_label(bar)

plt.tight_layout()
plt.show()
```



monthly sales trends

```
monthly_trends = df_train.groupby('purchase_month')['payment_value'].sum()
monthly_trends = monthly_trends.reset_index()
monthly_trends
```

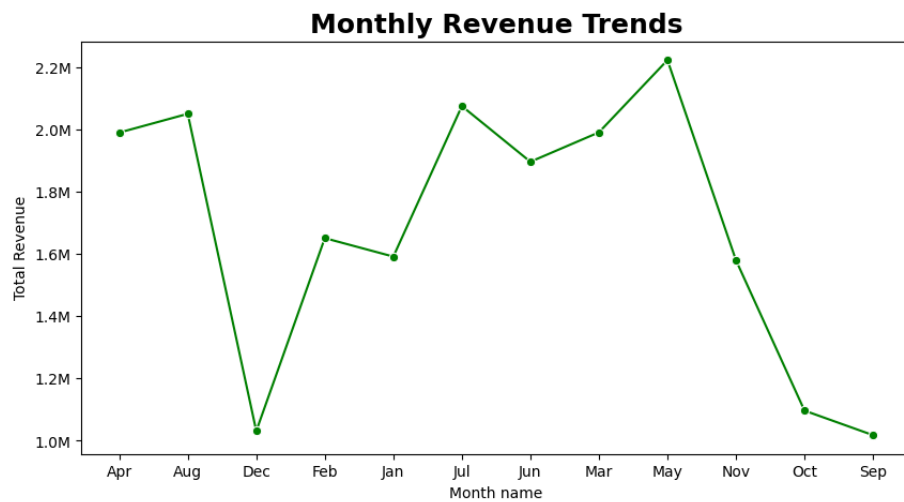
	purchase_month	payment_value
0	Apr	1989295.87
1	Aug	2049528.99
2	Dec	1031305.85
3	Feb	1650455.90
4	Jan	1591214.33
5	Jul	2074365.39
6	Jun	1895634.60
7	Mar	1990070.24
8	May	2221998.93
9	Nov	1579407.32
10	Oct	1096703.77
11	Sep	1017947.51

```
plt.figure(figsize=(10,5))
```

```
ax = sns.lineplot(x='purchase_month',y='payment_value',data=monthly_trends,marker='o',color=)
```

```
ax.yaxis.set_major_formatter(FuncFormatter(format_k))
```

```
plt.title("Monthly Revenue Trends",fontsize = 18, fontweight = 'bold')
plt.xlabel("Month name")
plt.ylabel("Total Revenue")
plt.show()
```



```
most_sale = df_train.groupby('customer_state')['payment_value'].sum().sort_values(ascending=False)
most_sale = most_sale.reset_index()
most_sale
```

	customer_state	payment_value
0	RR	12462.21
1	AP	21572.32
2	AC	24984.86
3	AM	34567.86
4	RO	65836.95
5	TO	72193.50
6	SE	86139.75
7	AL	110024.18
8	RN	115612.22
9	PI	135903.03

Top categories

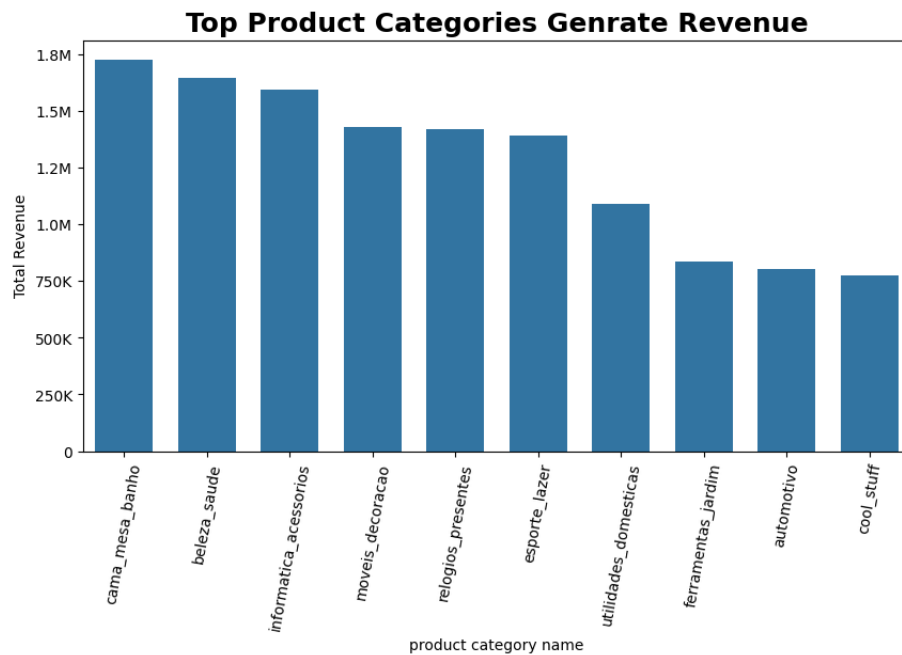
```
top_categories1 = df_train.groupby('product_category_name')['payment_value'].sum().sort_values(ascending=False)
top_categories1 = top_categories1.reset_index()
plt.figure(figsize=(10,5))

ax = sns.barplot(x='product_category_name',y='payment_value',data=top_categories1,width=0.7)

plt.xticks(rotation = 80)

ax.yaxis.set_major_formatter(FuncFormatter(format_k))
```

```
plt.title("Top Product Categories Genrate Revenue",fontsize = 18, fontweight = 'bold')
plt.xlabel("product category name")
plt.ylabel("Total Revenue")
plt.show()
```



Top sellers

```
top_sellers_state = df_train.groupby('seller_state')['payment_value'].sum().sort_values(ascending=False)
top_sellers_state = top_sellers_state.reset_index()
top_sellers_state
```

	seller_state	payment_value
0	SP	13286552.82
1	PR	1844070.05
2	MG	1550901.48
3	RJ	1090516.44
4	SC	883071.74
5	RS	558581.50
6	BA	366024.93
7	DF	137812.23
8	PE	122657.65

```

9          GO          112008.39

plt.figure(figsize=(10,5))

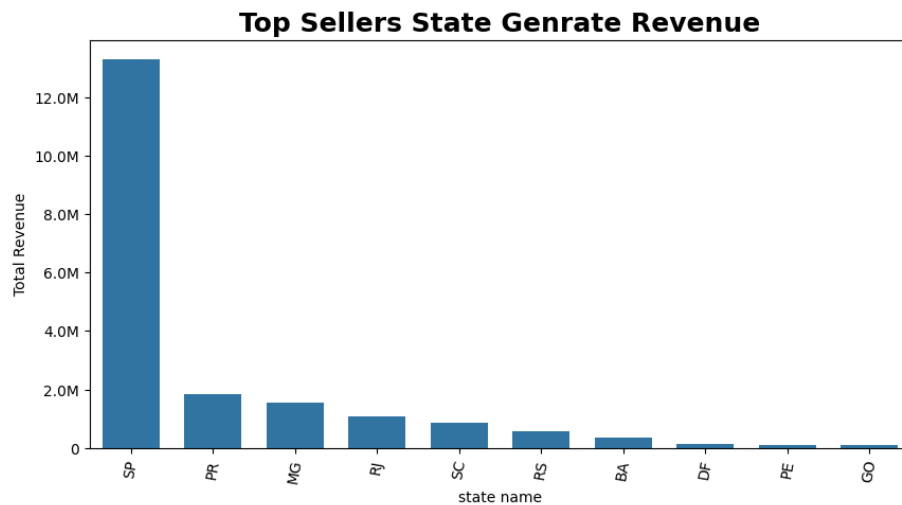
ax = sns.barplot(x='seller_state',y='payment_value',data=top_sellers_state,width=0.7)

plt.xticks(rotation = 80)

ax.yaxis.set_major_formatter(FuncFormatter(format_k))

plt.title("Top Sellers State Genrate Revenue",fontsize = 18, fontweight = 'bold')
plt.xlabel("state name")
plt.ylabel("Total Revenue")
plt.show()

```



customers Behavior (customer retactions)

```

df_train['customer_unique_id'].nunique()

94720

# how many customers repet agains

reppet_customer_order = df_train.groupby('customer_unique_id')['order_id'].nunique()

reppet_customer_order
customer_unique_id

```

```

0000366f3b9a7992bf8c76cfdf3221e2    1
0000b849f77a49e4a4ce2b2a4ca5be3f    1
0000f46a3911fa3c0805444483337064    1
0000f6ccb0745a6a4b88665a16c9f078    1
0004aac84e0df4da2b147fca70cf8255    1
..
ffffcf5a5ff07b0908bd4e2dbc735a684    1
ffffea47cd6d3cc0a88bd621562a9d061    1
fffff371b4d645b6ecea244b27531430a    1
fffff5962728ec6157033ef9805bacc48    1
fffffd2657e2aad2907e67c3e9daecbeb    1
Name: order_id, Length: 94720, dtype: int64

repet_customer = repet_customer_order[repet_customer_order>1].count()
repet_customer

np.int64(2869)

repet_rate = round((repet_customer/repet_customer_order.count())*100,2)
repet_rate

np.float64(3.03)

```

Average spending

```

average_spending = round(df_train.groupby('customer_id')['payment_value'].sum().mean(),2)
average_spending

np.float64(206.18)

```

customers segmentaions RFM analysis

```

rfm = df_train.groupby('customer_id').agg({
    'order_purchase_timestamp': 'max',
    'order_id': 'nunique',
    'payment_value': 'sum'
}).reset_index()

rfm.columns = [
    'customer_id',
    'purachases_date',
    'frequency',
    'monetray']

latest_date = df_train['order_purchase_timestamp'].max()

```



```

rfm['recency'] = (
    latest_date - rfm['purachases_date']).dt.days

def customer_segmentaions(row):
    if row['monetray'] > 50000:
        return 'High value'
    elif row['monetray'] > 25000:
        return 'Medium value'
    else:
        return 'Low Value'

rfm['segment'] = rfm.apply(customer_segmentaions,axis=1)
rfm['segment'].value_counts()

segment
Low Value      97911
Medium value      4
High value       1
Name: count, dtype: int64

```

customers satisifcations analysis

Review Score Analysis

```

df_train['review_score'].value_counts()

review_score
5.0    66264
4.0    22286
1.0    14854
3.0     9840
2.0     4085
Name: count, dtype: int64

plt.figure(figsize=(6,4))
ax = sns.countplot(
    x='review_score',
    data=df_train,
    color='yellow',
    width=0.6
)

# Title
plt.title(

```

```

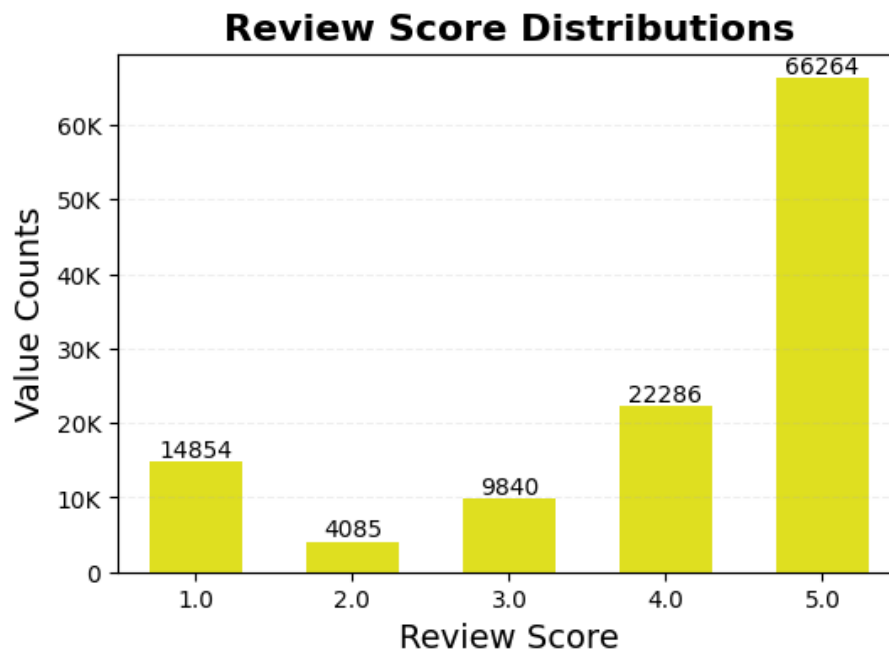
    "Review Score Distributions",
    fontsize = 16,
    fontweight = 'bold'
)
# Axis label
plt.xlabel('Review Score', fontsize = 14)
plt.ylabel('Value Counts', fontsize = 14)

ax.yaxis.set_major_formatter(FuncFormatter(format_k))

# grid
plt.grid(axis='y', linestyle = '--', alpha=0.2)

for bar in ax.containers:
    ax.bar_label(bar)
plt.show()

```



most customer are satisfied but low ratings indicate operations issues in creations order

Delivery Analysis

```
df_train['delivery_days'].describe()
```

```

count      114866.000000
mean        11.980177
std         9.371545
min         0.000000
25%         6.000000
50%        10.000000
75%        15.000000
max        208.000000
Name: delivery_days, dtype: float64

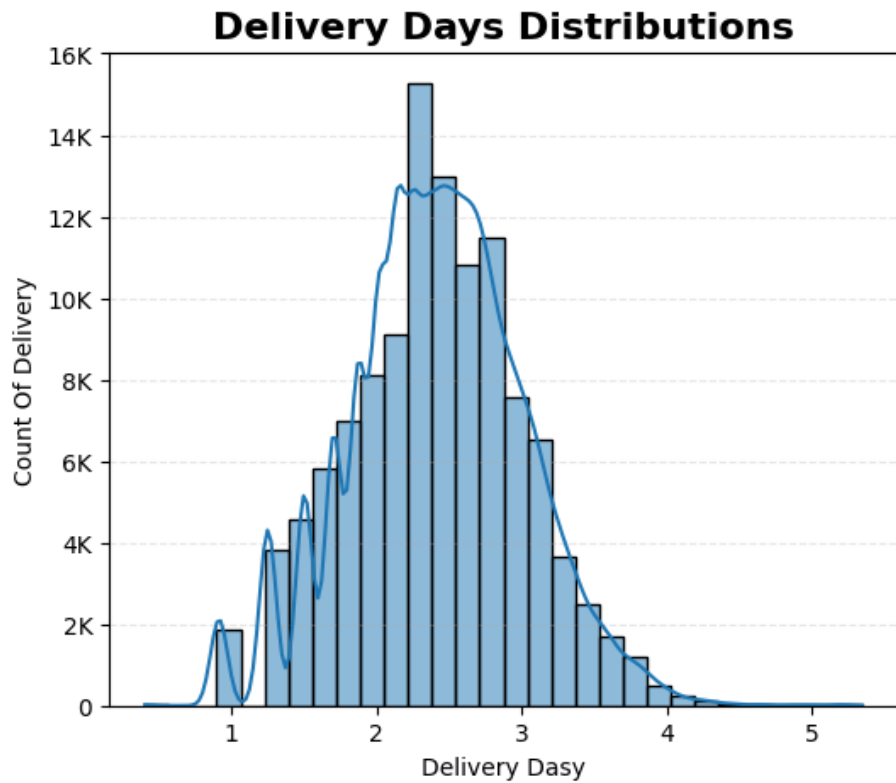
plt.figure(figsize=(6,5))
ax = sns.histplot(
    np.log(df_train['delivery_days']+1.5),
    bins=30,
    kde=True
)
plt.title('Delivery Days Distributions',fontsize = 16, fontweight = 'bold')
plt.xlabel('Delivery Dasy', fontsize=10)
plt.ylabel('Count Of Delivery', fontsize=10)

ax.yaxis.set_major_formatter(FuncFormatter(format_k))

plt.grid(axis='y',linestyle = '--',alpha=0.3)

plt.show()

```



most delivery are completed widthin 2-3 days acceptable timeline through some orders expreances extream dileay

Revenue Analysis

`df_train.columns`

```
Index(['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp',
      'order_approved_at', 'order_delivered_carrier_date',
      'order_delivered_customer_date', 'order_estimated_delivery_date',
      'order_item_id', 'product_id', 'seller_id', 'shipping_limit_date',
      'price', 'freight_value', 'product_category_name', 'product_weight_g',
      'product_length_cm', 'product_height_cm', 'product_width_cm',
      'payment_sequential', 'payment_type', 'payment_installments',
      'payment_value', 'customer_unique_id', 'customer_zip_code_prefix',
      'customer_city', 'customer_state', 'review_id', 'review_score',
      'review_creation_date', 'review_answer_timestamp',
      'seller_zip_code_prefix', 'seller_city', 'seller_state',
      'delivery_days', 'delivery_delay_days', 'delay_flag', 'purchase_month',
```

```

        'purchase_year'],
        dtype='object')

sellers_revenue = df_train.groupby('seller_id')['payment_value'].sum().sort_values(ascending=False)
sellers_revenue

seller_id
7c67e1448b00f6e969d365cea6b010ab    509474.13
1025f0e2d44d7041d6cf58b6550e0bfa    310579.23
4a3ca9315b744ce9f8e9374361493884    302403.81
1f50f920176fa81dab994f9023523100    290729.12
53243585a1d6dc2643021fd1853d8905    282750.15
da8622b14eb17ae2831f4ac5b9dab84a    275667.80
4869f7a5dfa277a7dca6462dcf3b52b2    262628.03
955fee9216a65b617aa5c0531780ce60    234603.88
fa1c13f2614d7b5c4749cbc52fecda94    205173.24
7e93a43ef30c4f03f38b393420bc753a    184867.29
Name: payment_value, dtype: float64

```

product quality issus

```

puroduct_issus = round(
    df_train.groupby('product_category_name')['review_score'].mean()
    .reset_index()
    .sort_values(by = 'review_score',ascending = False),1
).head(10)
puroduct_issus

   product_category_name  review_score
18          cds_dvds_musicais          4.6
38  fashion_roupa_infanto_juvenil          4.5
49      livros_interesse_geral          4.4
42                flores          4.4
48      livros_importados          4.4
23  construcao_ferramentas_ferramentas          4.4
50      livros_tecnicos          4.4
3          alimentos_bebidas          4.3
65  portateis_casa_forno_e_cafe          4.3
51      malas_acessorios          4.3

plt.figure(figsize=(10,5))

ax = sns.barplot(x='product_category_name',y='review_score', data=puroduct_issus, width=0.7)

plt.title(
    "Top Product Categories by Quality",
    fontsize=16,

```

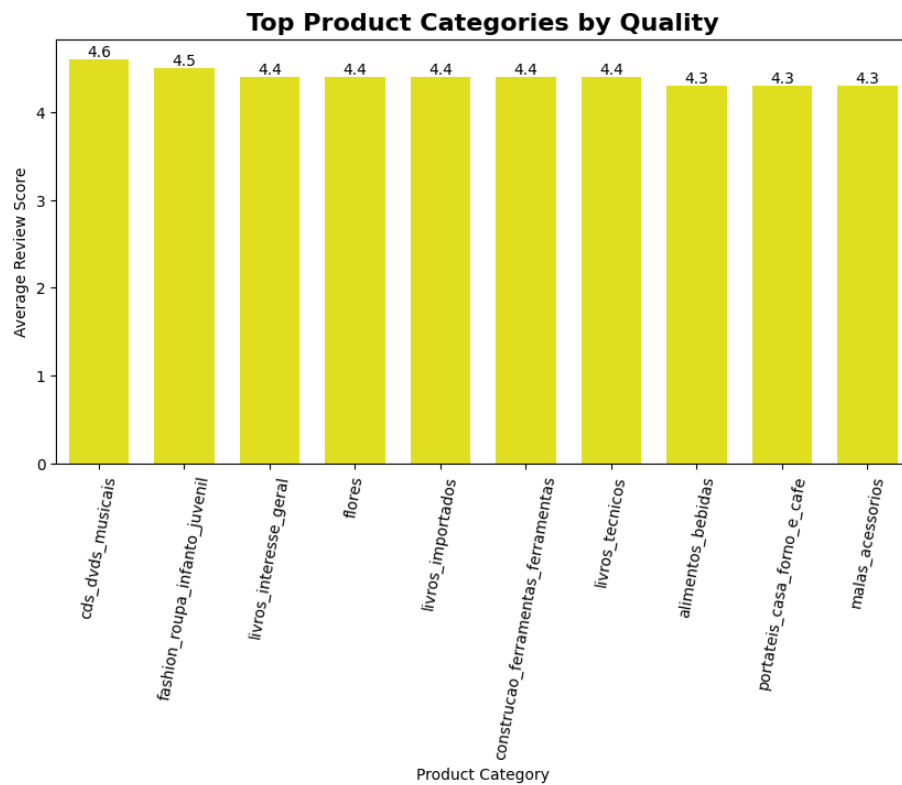
```

        fontweight='bold'
    )

    for bar in ax.containers:
        ax.bar_label(bar)
    plt.xticks(rotation = 80)

    plt.xlabel("Product Category")
    plt.ylabel("Average Review Score")
    plt.show()

```



Delivery Performance Analysis

Average Delivery Days

```

round(df_train['delivery_days'].mean())
12

```

```

is_delay_orders = df_train['delay_flag'].sum()
is_delay_orders

np.int64(8860)

state_delivery = round(df_train.groupby('customer_state')['delivery_days']
                        .mean()
                        .reset_index()
                        .sort_values(by='delivery_days',ascending = False )).head(10)
state_delivery

```

	customer_state	delivery_days
21	RR	28.0
3	AP	28.0
2	AM	26.0
1	AL	24.0
13	PA	23.0
9	MA	21.0
24	SE	21.0
5	CE	20.0
0	AC	20.0
14	PB	20.0

```

plt.figure(figsize=(6,5))

ax = sns.barplot(
    x='customer_state',
    y='delivery_days',
    data=state_delivery,
    width=0.7,
    color='tomato'
)

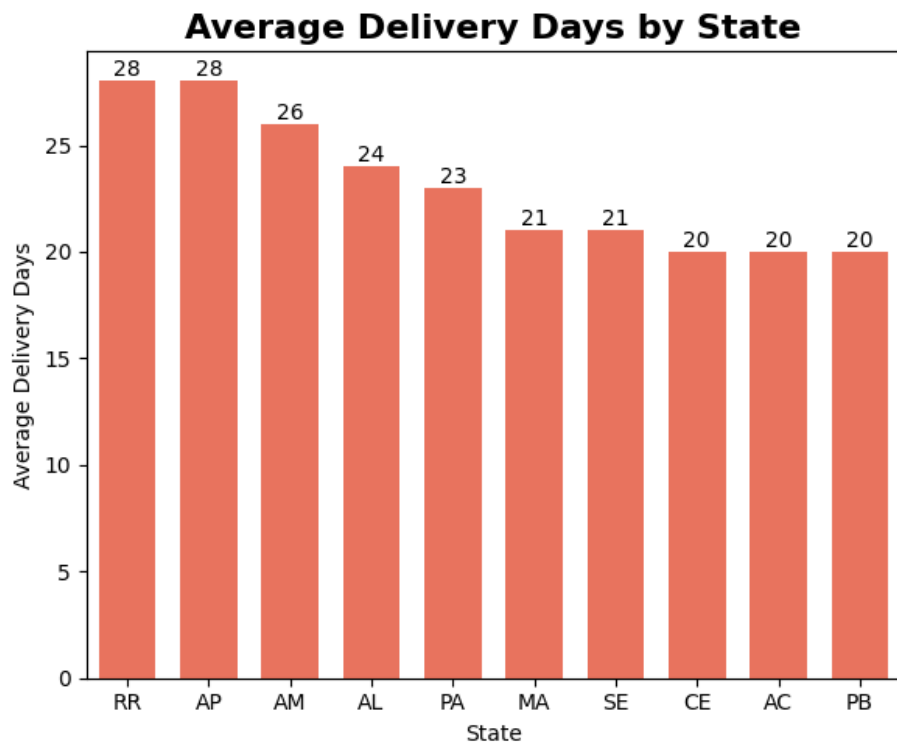
plt.title(
    "Average Delivery Days by State",
    fontsize=16,
    fontweight='bold'
)

plt.xlabel("State")
plt.ylabel("Average Delivery Days")

for i in ax.containers:
    ax.bar_label(i)

plt.tight_layout()
plt.show()

```



Estimate vs actual day delivery

```
df_train['actual_delivery_days'] = (
    df_train['order_delivered_customer_date'] - df_train['order_purchase_timestamp']).dt.days
df_train['estimated_delivery_days'] = (
    df_train['order_estimated_delivery_date'] - df_train['order_purchase_timestamp']).dt.days

df_train['delivery_difference'] = df_train['actual_delivery_days'] - df_train['estimated_delivery_days']

df_train['is_delay'] = (df_train['delivery_difference'] > 0)

df_train['is_delay'].sum()

np.int64(8268)

df_train.head()
```

	order_id	customer_id \
0	7f39ba4c9052be115350065d07583cac	d7fc82cbeafea77bd0a8fbbf6296e387
1	9dc8d1a6f16f1b89874c29c9d8d30447	d9442164acf4b03109425633efaa0cfc
2	d455a8cb295653b55abda06d434ab492	944b72539d7e1f7f7fc6e46639ef1fe3
3	006e43460a55bc60c0a437521e426529	23bfd4316e261786deed5a08231c75bc

4 00dfb074b5c910fbd08e04691c4b712f a5ced4926d7d8fa71e9be2b007720356

	order_status	order_purchase_timestamp	order_approved_at	\
0	delivered	2017-10-18 08:16:34	2017-10-18 23:56:20	
1	delivered	2017-10-12 13:33:22	2017-10-12 13:49:22	
2	delivered	2017-09-26 22:17:05	2017-09-27 22:24:16	
3	delivered	2017-05-11 00:24:35	2017-05-11 01:30:22	
4	delivered	2017-06-08 19:43:35	2017-06-08 19:55:19	

	order_delivered_carrier_date	order_delivered_customer_date	\
0	2017-10-20 14:29:01	2017-10-27 16:46:05	
1	2017-10-17 15:42:42	2017-10-24 20:17:44	
2	2017-09-29 15:53:03	2017-10-07 16:12:47	
3	2017-05-12 12:26:32	2017-05-19 09:51:17	
4	2017-06-09 15:12:41	2017-06-15 09:03:59	

	order_estimated_delivery_date	order_item_id	\
0	2017-11-09	1	
1	2017-11-06	1	
2	2017-10-30	1	
3	2017-06-02	1	
4	2017-07-10	1	

	product_id	...	seller_state	delivery_days	\
0	a2ff5a97bf95719e38ea2e3b4105bce8	...	SP	9.0	
1	a2ff5a97bf95719e38ea2e3b4105bce8	...	SP	12.0	
2	a2ff5a97bf95719e38ea2e3b4105bce8	...	SP	10.0	
3	08574b074924071f4e201e151b152b4e	...	ES	8.0	
4	4f3b83b83f7fb280fca4f90ba502f50e	...	ES	6.0	

	delivery_delay_days	delay_flag	purchase_month	purchase_year	\
0	-13.0	0	Oct	2017	
1	-13.0	0	Oct	2017	
2	-23.0	0	Sep	2017	
3	-14.0	0	May	2017	
4	-25.0	0	Jun	2017	

	actual_delivery_days	estimated_delivery_days	delivery_difference	\
0	9.0	21	-12.0	
1	12.0	24	-12.0	
2	10.0	33	-23.0	
3	8.0	21	-13.0	
4	6.0	31	-25.0	

	is_delay
0	False

```
1     False
2     False
3     False
4     False
```

```
[5 rows x 43 columns]
```

product Category analysos

```
df_train['product_category_name'].nunique()
```

```
74
```

```
df_train.columns
```

```
Index(['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp',
       'order_approved_at', 'order_delivered_carrier_date',
       'order_delivered_customer_date', 'order_estimated_delivery_date',
       'order_item_id', 'product_id', 'seller_id', 'shipping_limit_date',
       'price', 'freight_value', 'product_category_name', 'product_weight_g',
       'product_length_cm', 'product_height_cm', 'product_width_cm',
       'payment_sequential', 'payment_type', 'payment_installments',
       'payment_value', 'customer_unique_id', 'customer_zip_code_prefix',
       'customer_city', 'customer_state', 'review_id', 'review_score',
       'review_creation_date', 'review_answer_timestamp',
       'seller_zip_code_prefix', 'seller_city', 'seller_state',
       'delivery_days', 'delivery_delay_days', 'delay_flag', 'purchase_month',
       'purchase_year', 'actual_delivery_days', 'estimated_delivery_days',
       'delivery_difference', 'is_delay'],
      dtype='object')
```

category demand

```
category_demand = (
    df_train.groupby('product_category_name')['order_id']
        .nunique()
        .reset_index()
        .sort_values(by= 'order_id', ascending = False)
).head(10)
category_demand
```

	product_category_name	order_id
14	cama_mesa_banho	9313
12	beleza_saude	8770
33	esporte_lazer	7669
45	informatica_acessorios	6649

```

55         moveis_decoracao      6398
73     utilidades_domesticas     5843
67         relorios_presentes    5576
71             telefonia         4168
9             automotivo         3877
13         brinquedos           3853

```

```
plt.figure(figsize=(14,6))
```

```

sns.barplot(
    x='product_category_name',
    y='order_id',
    data=category_demand,
    width=0.7,
    color='green'
)

```

```

plt.title(
    "Top Product Categories by Demand",
    fontsize=16,
    fontweight='bold'
)

```

```

plt.xlabel("Product Category")
plt.ylabel("Total Orders")

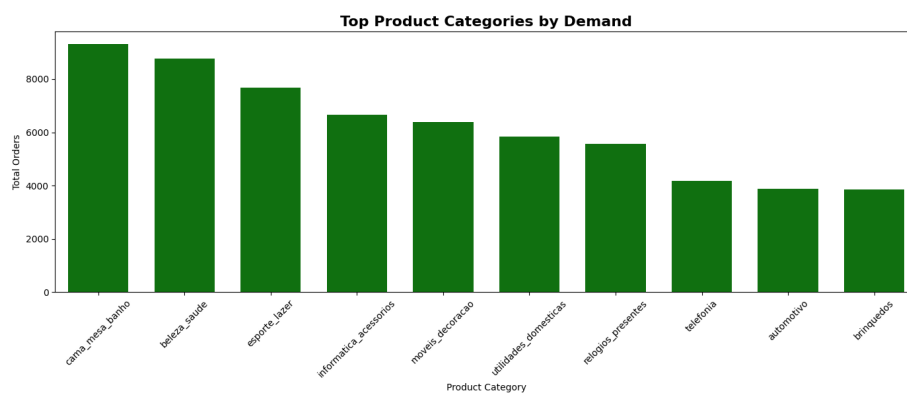
```

```
plt.xticks(rotation=45)
```

```

plt.tight_layout()
plt.show()

```



Root cause analysis (RAC)

```
finding the main reason bi
round(df_train['review_score'].mean(),1)
np.float64(4.0)
df_train.groupby('is_delay')['review_score'].mean()
is_delay
False    4.151832
True     2.443759
Name: review_score, dtype: float64
```

ETL Pipeline

```
# create sql connections
engine = create_engine(
    "mysql+pymysql://root:admin@localhost:3306/retail_sales"
)

df_train.to_sql(
    name='df_train',
    con=engine,
    if_exists='replace',
    index=False
)

117329
```